UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DCC 703 – COMPUTAÇÃO GRÁFICA
PROF DR LUCIANO F. SILVA

ANDREZA OLIVEIRA GONÇALVES

RASTERIZAÇÃO DA CURVA DE BÉZIER

Relatório de implementação e resultados de algoritmos de curva de Bézier

Boa Vista/RR,
outubro de 2026

RASTERIZAÇÃO DE CURVAS **1**
1. Introdução e Objetivos **3**
2. Fundamentação Teórica **3**
Método da Equação Paramétrica 3
Algoritmo de Casteljau 4
3. Implementação **5**
4. Resultados **5**
5. Conclusão **6**

1. Introdução e Objetivos

A Curva de Bézier é amplamente utilizada em computação gráfica, animações, design gráfico e modelagem de superfícies. Essa curva é definida por um conjunto de pontos de controle, que influenciam sua forma. O objetivo deste relatório é construir e comparar duas formas diferentes de gerar a curva de Bézier:

- Equação Paramétrica;
- Algoritmo de *Casteljau*.

2. Fundamentação Teórica

A Curva de Bézier é definida pela seguinte equação, baseada nos Polinômios de Bernstein:

$$B_{i,n}(t) = \sum_{i=0}^{n} P_i(n\,i)(1 - t)^{n-1} t^i$$

Onde:

- Bi,n(t) é o ponto na curva para um valor t.
- Pi são os pontos de controle.
- (ni) é o coeficiente binomial.
- t varia de 0 a 1.

Em uma curva de Bézier, as coordenadas P(x,y) inicial e final ancoram o início e o fim da linha, enquanto P(x,y) intermediários atuam como "ímãs" geométricos fora da curva. À medida que o parâmetro tempo t avança de 0 a 1, o cálculo com o polinômios de Bernstein interpola esses pontos, gerando uma trajetória contínua que é puxada pela gravidade dos ímãs sem nunca encostar neles. Para entender como funcionam os programas, é necessário entender melhor os métodos:

Método da Equação Paramétrica

A equação paramétrica calcula diretamente os pontos da curva de Bézier a partir dos pontos de controle. O grau da curva é determinado pelo número de pontos de controle:

- Grau 1 → Linha reta (Interpolação linear).
- Grau 2 → Curva Quadrática.
- Grau 3 → Curva Cúbica.
- Grau n → Curva de Bézier de ordem superior.

Eis o funcionamento do algoritmo:

- **Definição paramétrica:** O algoritmo parte de um par de coordenadas P(x,y) e os pontos intermediários que podem ser um ou dois, o qual formam um conjunto de

pontos de controle. No caso de três, configura-se uma curva quadrática, e no caso de quatro, uma curva cúbica. Esse algoritmo isola a curva no "tempo" utilizando a variável independente t, que avança estritamente no intervalo fechado [0, 1].

- **Polinômio de Bernstein:** Para cada valor de t, o algoritmo precisa descobrir qual a o peso percentual que cada ponto de controle exerce sobre a curva naquele exato milissegundo. Essa força é calculada pela fórmula dos Polinômios de Bernstein: Bi,n(t) onde n é o grau da curva (número de pontos menos 1) e i é o índice do ponto atual.
- **Soma vetorial:** Com os pesos de atração já descobertos, o algoritmo executa a aplicação geométrica final. Ele multiplica as coordenadas cartesianas (x,y) de cada ponto de controle Pi pelo seu respectivo peso de Bernstein, somando tudo numa única coordenada resultante C(t):

$$C(t) = \sum_{i=0}^{n} B_{i,n}(t) P_i$$

Curva de Bézier Quadrática (Grau 2)

Uma curva de Bézier quadrática é definida por três pontos de controle: P0, P1 e P2. Com isso, a soma resultante C(t) é arranjada da seguinte maneira:

$$C(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2$$

Curva de Bézier Cúbica (Grau 3)

Uma curva de Bézier cúbica é definida por quatro pontos de controle: P0, P1, P2 e P3. Com isso, a soma resultante C(t) é arranjada da seguinte maneira:

$$C(t) = (1 - t)^3 P_0 + 3t(1 - t)^2 P_1 + 3t^2(1 - t)P_2 + t^3 P_3$$

Algoritmo de *Casteljau*

O método de Casteljau substitui as operações exponenciais por interpolações lineares recursivas e garante estabilidade numérica contra erros de arredondamento. Eis o funcionamento deste método:

- **Interpolação linear recursiva:** O algoritmo avalia a curva para um parâmetro t em [0, 1] realizando interpolações lineares sucessivas entre pontos de controle consecutivos (P0, P1, ..., Pn). Cada etapa reduz o número de pontos intermediários em um até obter um único ponto final sobre a curva.
- **Caso Quadrático (3 pontos de controle - Grau 2):** Para os pontos P0, P1 e P2, calcula-se primeiro dois pontos intermediários A = (1-t)P0 + tP1 e B = (1-t)P1 + tP2. Em seguida, intercala-se entre A e B para obter o ponto final da curva: C(t) = (1-t)A + tB.

- **Caso Cúbico (4 pontos de controle - Grau 3):** Para os pontos P0, P1, P2 e P3, a primeira etapa gera três pontos intermediários (A, B, C) entre os pares consecutivos. A segunda etapa interpola esses três pontos para gerar dois novos pontos (D, E). A terceira e última etapa interpola D e E para encontrar o ponto pertencente à curva cúbica.
- **Subdivisão geométrica:** Além de calcular pontos individuais na curva, a estrutura de pontos intermediários gerada pelo algoritmo de Casteljau permite subdividir uma curva de Bézier em duas sub curvas independentes da mesma ordem, facilitando o renderizador geométrico e testes de intersecção.

Essa capacidade de divisão recursiva proporciona extrema eficiência no traçado e na renderização adaptativa, permitindo interromper o refinamento assim que os segmentos atingirem a precisão visual desejada ou a escala de pixels do dispositivo. Além disso, ao evitar o cálculo direto de potências polinomiais, o algoritmo assegura alta estabilidade numérica contra erros de arredondamento em ponto flutuante.

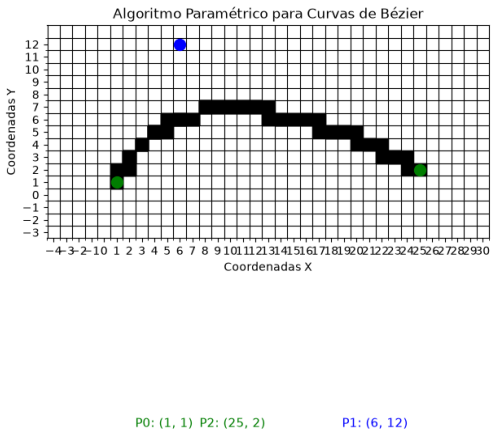
3. Implementação

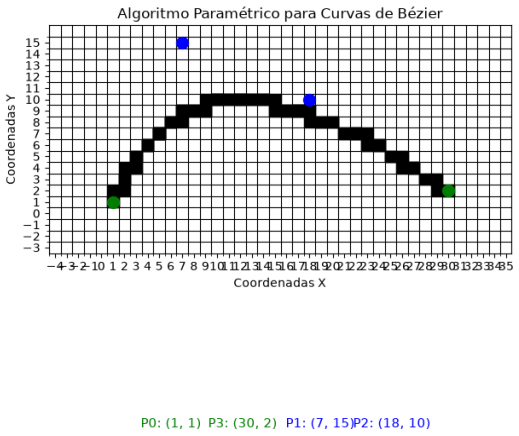
- **Linguagem e ambiente:** O desenvolvimento foi estruturado na linguagem Python, utilizando as bibliotecas *NumPy* e *Matplotlib* para a plotagem e representação visual da grade discreta de pixels.

4. Resultados

No resultado da curva de Bézier por algoritmo paramétrico, a curva cúbica segue bem os pontos de controle, mas teve um encontro com um deles no caso da Figura 2.

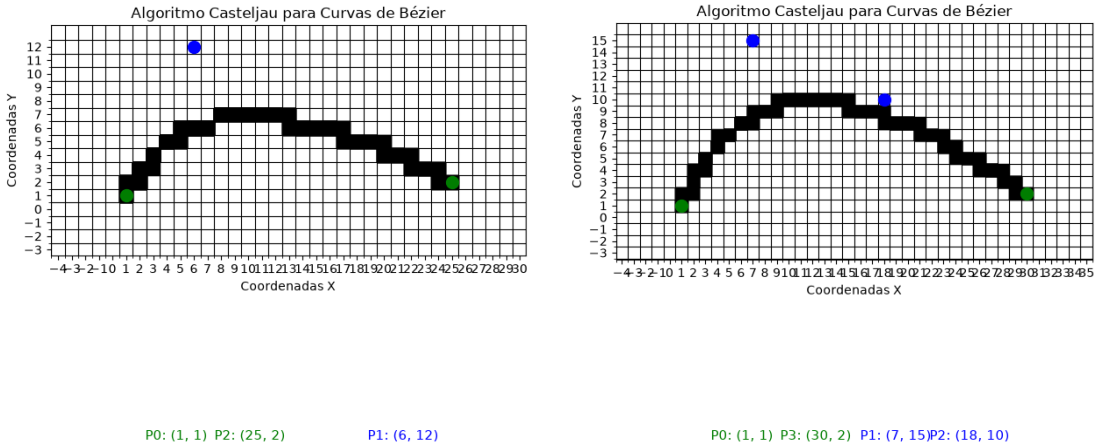
Figuras 1 e 2 – Desenhos da curvas de Bézier quadrática e cúbica pelo algoritmo Paramétrico





Fonte: Autoria própria, 2026

Figuras 3 e 4 – Desenhos da curvas de Bézier quadrática e cúbica pelo algoritmo *Casteljau*



Fonte: Autoria própria, 2026

A análise comparativa entre a representação paramétrica e o algoritmo de *Casteljau* demonstra que, embora ambos gerem a mesma geometria final, eles representam abordagens bem distintas na computação gráfica. O modelo paramétrico, fundamentado nos polinômios de Bernstein, entrega uma resolução algébrica direta. Em contrapartida, o algoritmo de *Casteljau* traduz a curva para a linguagem nativa do processador. Ao substituir as operações exponenciais por interpolações lineares recursivas, ele garante estabilidade numérica contra erros de arredondamento e otimiza o ciclo de máquina.

5. Conclusão

Na avaliação dos resultados, a escolha entre os dois não reside na qualidade do traço, mas no custo arquitetural: a forma paramétrica ganha na modelagem teórica e animação, enquanto a de Casteljau é a base algorítmica essencial para a performance e estabilidade do hardware gráfico, o que impacta completamente o custo da recursão, entregando uma velocidade de processamento significativamente superior e garantindo estabilidade para figuras de qualquer magnitude.

Embora a abordagem paramétrica seja ideal para o controle teórico e o sincronismo em animações ao mapear com exatidão a curva no tempo, o seu uso de potências flutuantes elevadas e funções contínuas gera gargalos computacionais inviáveis para renderizações em tempo real. Em contrapartida, o método de *Casteljau* oferece a vantagem de uma subdivisão geométrica natural, simplificando curvas complexas por meio de sua fragmentação em segmentos menores, o que possibilita a aplicação otimizada de algoritmos baseados em intersecção, facilitando consideravelmente as tarefas de renderização e processamento gráfico.